

best tech stack for a SOC Hub dashboard webpage, showing key metrics across different components, system health monitors, uptime and a shortcut menu to the different admin pages along with keycloak sso



 Grafana Labs — Keycloak Metrics Da

 GExperts — Enabling metrics in RH-5

 metabase — Dashboard

For a **SOC Hub dashboard** — think a centralized “mission control” for your security and systems telemetry — you want a tech stack that is:

1. **Secure and SSO-friendly** (integrates smoothly with Keycloak).
2. **Reactive and modular** (so dashboards update live).
3. **Extensible** (so you can plug in metrics from Wazuh, Zabbix, 3CX, etc.).
4. **Stable for 24/7 uptime** with minimal bloat.

Here’s a best-in-class setup for a modern SOC Hub, with a balance between enterprise robustness and open-source flexibility:

## Core Stack Overview

### Frontend (Dashboard UI)

- **Framework:** React + TypeScript (fast, modular, excellent for dynamic dashboards)
- **UI Library:** MUI (Material UI) or ShadCN + TailwindCSS for a clean admin look
- **Charts & Metrics:** Recharts, ECharts, or ApexCharts for visual metrics
- **Live Data:** React Query or SWR for live refresh, WebSockets for push events
- **Auth Integration:** Keycloak’s OIDC JS Adapter for session & role-based UI control
- **Routing:** React Router or Next.js built-in routing if using SSR

### Optional:

If you prefer prebuilt dashboard controls, **Grafana’s frontend SDK** can embed directly in a React app — useful if you already have metrics in Prometheus or Elasticsearch.

### Backend (API + Data Orchestration Layer)

- **Runtime:** Node.js (NestJS) or FastAPI (Python) — both handle structured APIs cleanly.
  - **NestJS** if you want integrated structure and decorators for RBAC, JWT, etc.
  - **FastAPI** if your team is Python-heavy and you’re pulling from security telemetry.
- **Auth & SSO:** Integrate **Keycloak OpenID Connect** — token verification middleware in Node/Python.
- **Real-time Data:** **Socket.IO** (Node) or **FastAPI-WebSocket** layer for live event streams.
- **Database:**
  - **PostgreSQL** for relational metrics, configs, audit logs.
  - **TimescaleDB** or **InfluxDB** for time-series (uptime, system health, etc.).
- **Cache/Queue:**
  - **Redis** for fast caching and background tasks (e.g., alert queueing).
- **API Aggregation Layer:**
  - Use **GraphQL (Apollo Server)** or **REST with OpenAPI spec** to unify your components (e.g., Wazuh, uptime monitors, Zabbix, etc.) into one clean endpoint.

### Monitoring & Metrics Sources

- **Prometheus + Node Exporter** — for server/system metrics
- **Wazuh** — for endpoint and SIEM feeds
- **Zabbix / Netdata / Uptime Kuma** — for uptime and service health
- **Elasticsearch** — for logs and threat data
- **Grafana / Loki / Tempo** — for visualization, log correlation, and tracing if you go full observability stack

All these can feed into your backend or be proxied through APIs that the dashboard polls.

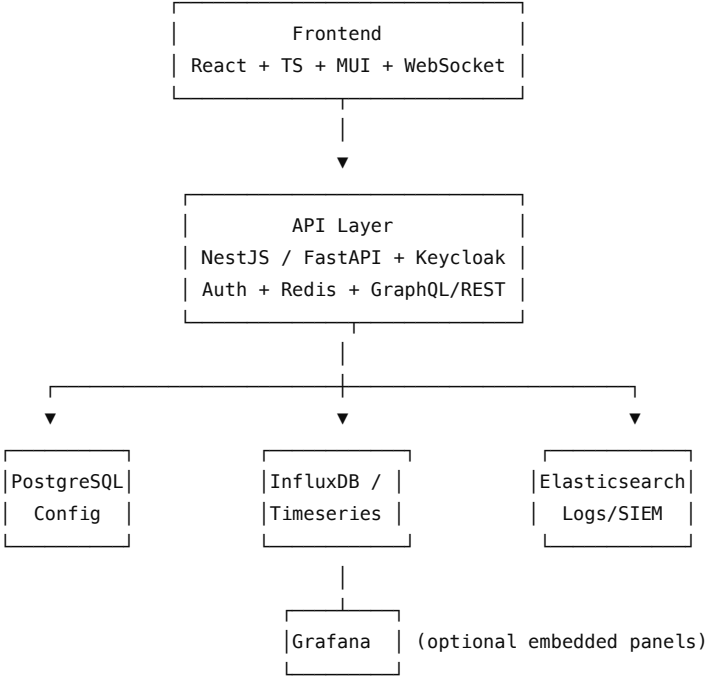
Security & SSO

- **Keycloak** — single source of truth for all dashboard logins.
  - Set up **roles (viewer/admin/SOC lead)** and use **role-based UI visibility** in frontend.
  - Integrate **Keycloak Gatekeeper** or use **Nginx reverse proxy** with Keycloak auth for sub-apps (Zabbix, Wazuh, Grafana).
- **HTTPS**: Terminate SSL at **Nginx/Traefik** reverse proxy.
- **CSP + CORS policies**: Strict, especially if you embed external dashboards.

🧩 Architecture Summary

cpp

📄 Copy code



🚀 Deployment

- **Docker Compose / Kubernetes** for service orchestration.
- **Traefik or Nginx** as reverse proxy + SSL termination.
- **CI/CD**: GitHub Actions or GitLab CI → automatic redeploys on push.
- **Telemetry**: Prometheus exporters + Grafana Cloud for external view.
- **Runtime Security**: Falco or CrowdSec for SOC-grade runtime event detection.

🧠 Recommended Stack Summary

| Layer         | Tool                  | Reason                       |
|---------------|-----------------------|------------------------------|
| Frontend      | React + MUI/Tailwind  | Fast, modular UI             |
| Charts        | Recharts / ApexCharts | Clean metric visuals         |
| Backend       | NestJS / FastAPI      | API orchestration, WebSocket |
| Auth          | Keycloak              | SSO + RBAC                   |
| DB            | PostgreSQL + InfluxDB | Config + time-series         |
| Cache         | Redis                 | Fast data & queues           |
| Observability | Prometheus + Grafana  | Metrics + logs               |
| Proxy         | Traefik / Nginx       | Routing + SSL                |
| Container     | Docker / K8s          | Portability & scaling        |

If you want, I can sketch the **directory structure** and **Keycloak integration pattern** (JWT flow + refresh tokens + role mapping) for either the **NestJS** or **FastAPI** version — which backend language would you

prefer for this project?

     ...

yes please

Excellent — let's do this properly.

We'll map both back-end options side-by-side so you can pick the one that fits your workflow best. Both use Keycloak for SSO and expose clean, modern APIs to feed your SOC dashboard UI.

## 1. NestJS (Node.js / TypeScript)

Perfect if you already use JavaScript in the frontend and want an enterprise-style modular API.

### Project Structure

lua

 Copy code

```
soc-hub/
├─ apps/
│   └─ dashboard/          # React frontend
├─ backend/
│   └─ src/
│       └─ main.ts         # App bootstrap
│       └─ app.module.ts
│       └─ auth/
│           └─ keycloak.guard.ts
│           └─ keycloak.strategy.ts
│       └─ metrics/
│           └─ metrics.controller.ts
│           └─ metrics.service.ts
│       └─ health/
│           └─ health.controller.ts
│       └─ config/
│           └─ config.module.ts
│       └─ Dockerfile
│       └─ package.json
├─ docker-compose.yml
└─ traefik.yml
```

### Keycloak Integration

In NestJS you can rely on the official `@nestjs/keycloak-connect` package:

ts

 Copy code

```
// app.module.ts
import { KeycloakConnectModule, ResourceGuard } from 'nest-keycloak-connect';

@Module({
  imports: [
    KeycloakConnectModule.register({
      authServerUrl: 'https://keycloak.example.com',
      realm: 'SOC',
      clientId: 'soc-hub-backend',
      secret: process.env.KEYCLOAK_SECRET,
    }),
  ],
  providers: [ResourceGuard],
})
export class AppModule {}
```

Controllers are then protected declaratively:

ts

 Copy code

```
@Controller('metrics')
@Roles('soc_viewer')
export class MetricsController {}
```

```
@Get()
findAll() { /* return current system metrics */ }
}
```

## Data & Realtime Layer

- Use **TypeORM** (PostgreSQL) for configs/audit logs.
- Use **WebSocketGateway** or **Socket.IO** for live events.
- Add **BullMQ** (Redis) for queued background tasks.

## 🐍 2. FastAPI (Python)

Ideal for security-oriented teams that want lightweight async APIs and Python ecosystem tooling.

### Project Structure

pgsql

 Copy code

```
soc-hub/
├── backend/
│   ├── main.py
│   ├── auth/
│   │   ├── keycloak_client.py
│   │   └── dependencies.py
│   ├── routers/
│   │   ├── metrics.py
│   │   ├── health.py
│   │   └── system.py
│   ├── models.py
│   ├── database.py
│   └── requirements.txt
└── docker-compose.yml
```

### Keycloak Integration

python

 Copy code

```
from fastapi import Depends, FastAPI, HTTPException
from fastapi_keycloak import FastAPIKeycloak

app = FastAPI()

keycloak = FastAPIKeycloak(
    server_url="https://keycloak.example.com/",
    client_id="soc-hub-backend",
    client_secret="YOUR_SECRET",
    admin_client_secret="YOUR_ADMIN_SECRET",
    realm="SOC",
    callback_uri="https://soc.example.com/callback"
)

keycloak.add_swagger_config(app)

@app.get("/metrics", dependencies=[Depends(keycloak.protect(role="soc_viewer"))])
def get_metrics():
    return {"uptime": 99.97, "alerts": 2}
```

### Data Layer

- Use **SQLModel** or **SQLAlchemy** for PostgreSQL models.
- Plug in **InfluxDB** or **Prometheus** clients for time-series pulls.
- Add **Redis** or **Celery** for background jobs.
- Include **uvicorn + gunicorn** in Docker for robust async serving.

## 🔑 Authentication Flow

1. User logs into **Keycloak** from the frontend ( /auth redirect).
2. Frontend receives `access_token` and `refresh_token`.
3. API requests include `Authorization: Bearer <token>`.
4. Backend middleware verifies token with Keycloak's public key.

5. Roles and permissions are decoded and matched against endpoint decorators.
6. Tokens are refreshed silently before expiry.

This pattern works identically in both NestJS and FastAPI.

⚡ Frontend (React Example)

tsx

Copy code

```
import Keycloak from 'keycloak-js';

const keycloak = new Keycloak({
  url: 'https://keycloak.example.com',
  realm: 'SOC',
  clientId: 'soc-hub-frontend'
});

keycloak.init({ onLoad: 'login-required' }).then(auth => {
  if (auth) {
    fetch('/api/metrics', {
      headers: { Authorization: `Bearer ${keycloak.token}` }
    });
  }
});
```

🚀 Suggested Deployment Stack

- **Frontend:** React served via Nginx container
- **Backend:** NestJS or FastAPI container
- **DB:** PostgreSQL + InfluxDB
- **Cache/Queue:** Redis
- **Reverse Proxy / SSL:** Traefik
- **Auth:** Keycloak
- **Monitoring:** Prometheus + Grafana
- **Logging:** Loki + Fluent Bit

All services orchestrated via Docker Compose or Helm on Kubernetes.

If you tell me which language you'll run your API in (Node or Python), I can generate the **production-grade Docker Compose setup** and the **frontend-to-Keycloak OIDC config** next. Which way are you leaning?



Node, I just need a full recommendaton and technical markdown document that outlines best approach

Perfect. Below is a **complete technical architecture document** in Markdown format — ready for your internal documentation or GitHub repo.

It defines a full **Node.js (NestJS)–based SOC Hub Dashboard stack**, integrated with **Keycloak SSO**, **PostgreSQL**, **InfluxDB**, and **Prometheus/Grafana**, designed for scalability, security, and observability.

markdown

Copy code

```
# 🚀 SOC Hub Dashboard – Technical Architecture & Implementation Plan

## Overview
The SOC Hub Dashboard provides a unified web interface for visualizing key metrics across

---

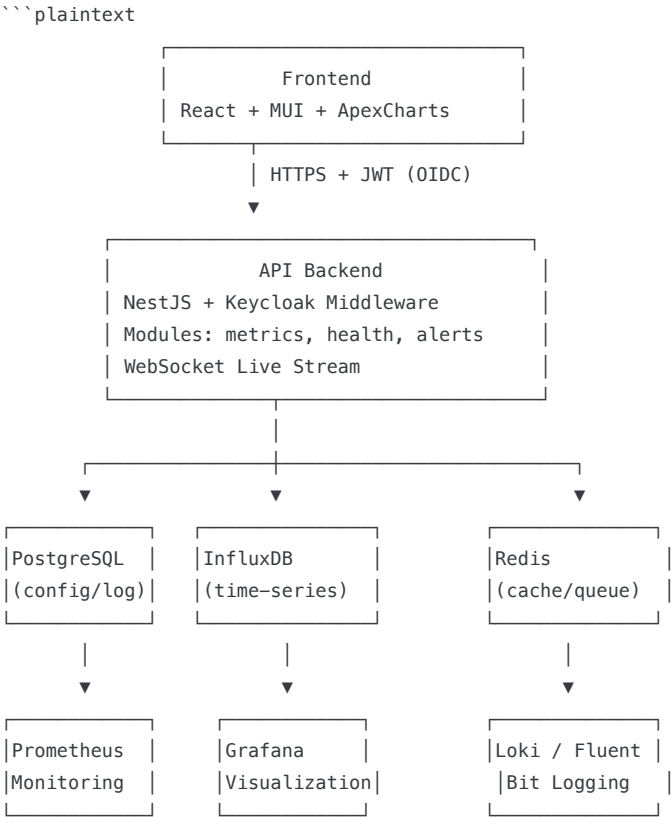
## 1. Objectives
- Centralized Visibility: Unified dashboard for SOC and IT operations.
- Security: All access controlled via Keycloak.
```

- **Extensibility:** API-driven backend with modular plugin design.
- **Realtime Insight:** WebSocket live updates for system health and alerts.
- **Scalability:** Microservice-ready architecture with container orchestration.

## 2. Core Technology Stack

| Layer            | Technology                            | Purpose                                           |
|------------------|---------------------------------------|---------------------------------------------------|
| Frontend         | React (TypeScript) + MUI + ApexCharts | Interactive dashboard & metrics visualization     |
| Auth             | Keycloak (OIDC)                       | Authentication, SSO, RBAC                         |
| API Backend      | NestJS (Node.js / TypeScript)         | REST + WebSocket data orchestration               |
| Database         | PostgreSQL                            | Configuration, logs, user preferences             |
| Time-Series DB   | InfluxDB                              | Performance, uptime & telemetry metrics           |
| Cache / Queue    | Redis                                 | Session caching, background tasks, message queues |
| Monitoring       | Prometheus + Grafana                  | Backend metrics & visualizations                  |
| Reverse Proxy    | Traefik                               | SSL termination & routing                         |
| Logging          | Loki + Fluent Bit                     | Centralized log aggregation                       |
| Containerization | Docker Compose / Kubernetes           | Deployment and scaling                            |

## 3. System Architecture



## 4. Authentication and Authorization

### Keycloak Integration

- **Realm:** soc
- **Clients:**
  - soc-hub-frontend (public, implicit flow)
  - soc-hub-backend (confidential, bearer-only)
- **Roles:**
  - soc\_viewer : read-only dashboards
  - soc\_admin : manage dashboards and settings
  - soc\_manager : access audit and reports

### Backend Setup

```
// app.module.ts
import { Module } from '@nestjs/common';
import { KeycloakConnectModule, ResourceGuard } from 'nest-keycloak-connect';

@Module({
  imports: [
    KeycloakConnectModule.register({
      authServerUrl: 'https://auth.example.com',
      realm: 'SOC',
      clientId: 'soc-hub-backend',
      secret: process.env.KEYCLOAK_SECRET,
    }),
  ],
  providers: [ResourceGuard],
})
export class AppModule {}
```

## Protected Endpoint Example

typescript

 Copy code

```
@Controller('metrics')
@UseGuards(AuthGuard, ResourceGuard)
@Roles('soc_viewer')
export class MetricsController {
  @Get('overview')
  getOverview() {
    return this.metricsService.getSystemMetrics();
  }
}
```

## 5. Backend Structure (NestJS)

lua

 Copy code

```
backend/
├── src/
│   ├── main.ts
│   ├── app.module.ts
│   ├── auth/
│   │   ├── keycloak.strategy.ts
│   │   └── keycloak.guard.ts
│   ├── metrics/
│   │   ├── metrics.controller.ts
│   │   ├── metrics.service.ts
│   │   └── metrics.gateway.ts
│   ├── health/
│   │   ├── health.controller.ts
│   │   └── health.service.ts
│   ├── alerts/
│   │   ├── alerts.controller.ts
│   │   └── alerts.service.ts
│   ├── config/
│   │   └── config.module.ts
│   └── common/
│       └── decorators/
├── package.json
└── Dockerfile
```

## 6. Frontend (React + TypeScript)

### Authentication Flow

- The frontend uses the `keycloak-js` adapter.
- Users are redirected to Keycloak for login.
- JWT token is stored in memory (never localStorage).
- Token refreshes silently via Keycloak session.

 Copy code

tsx

```
import Keycloak from 'keycloak-js';

const keycloak = new Keycloak({
  url: 'https://auth.example.com',
  realm: 'SOC',
  clientId: 'soc-hub-frontend'
});

keycloak.init({ onLoad: 'login-required' }).then(authenticated => {
  if (authenticated) {
    fetch('/api/metrics/overview', {
      headers: { Authorization: `Bearer ${keycloak.token}` }
    });
  }
});
```

### Frontend Directory Example

css

```
frontend/
├── src/
│   ├── components/
│   │   ├── charts/
│   │   ├── sidebar/
│   │   └── cards/
│   ├── pages/
│   │   ├── dashboard.tsx
│   │   ├── health.tsx
│   │   └── alerts.tsx
│   ├── hooks/
│   ├── api/
│   ├── App.tsx
│   └── main.tsx
└── package.json
```

Copy code

## 7. Data Sources and Integrations

| Source               | Method            | Example Use                   |
|----------------------|-------------------|-------------------------------|
| Wazuh                | REST API / Syslog | Security events, agent status |
| Prometheus           | HTTP API          | Node exporter metrics         |
| InfluxDB             | Native client     | System performance history    |
| Zabbix / Uptime Kuma | REST API          | Availability checks           |
| Loki                 | Log aggregation   | Event correlation             |

## 8. Realtime Event Handling

Use **WebSockets (Socket.IO)** in NestJS for pushing updates:

typescript

```
@WebSocketGateway({ cors: true })
export class MetricsGateway {
  @WebSocketServer() server: Server;

  handleMetricUpdate(data) {
    this.server.emit('metric_update', data);
  }
}
```

Copy code

The frontend listens via a lightweight React hook:

tsx

Copy code



```
useEffect(() => {
  const socket = io('/api');
  socket.on('metric_update', setData);
  return () => socket.disconnect();
}, []);
```

## 9. Deployment Model

### Docker Compose (Core Services)

yaml

 Copy code

```
version: '3.9'
services:
  keycloak:
    image: quay.io/keycloak/keycloak:latest
    command: start-dev
    environment:
      KEYCLOAK_ADMIN: admin
      KEYCLOAK_ADMIN_PASSWORD: admin
    ports:
      - 8080:8080

  backend:
    build: ./backend
    environment:
      DATABASE_URL: postgres://user:pass@db:5432/soc
      KEYCLOAK_SECRET: your_secret
    depends_on:
      - db
      - keycloak

  frontend:
    build: ./frontend
    ports:
      - 3000:80
    depends_on:
      - backend

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: soc

  influxdb:
    image: influxdb:2
    ports:
      - 8086:8086

  redis:
    image: redis:latest

  traefik:
    image: traefik:v2.10
    command:
      - "--providers.docker=true"
      - "--entrypoints.websecure.address=:443"
    ports:
      - 80:80
      - 443:443
```

## 10. Security Recommendations

- Enforce HTTPS everywhere (Traefik with Let's Encrypt).
- Configure CORS whitelist per domain.
- Use short-lived tokens with refresh cycles.

- Log all API access in PostgreSQL audit table.
- Store secrets in Docker secrets or Vault.
- Enable mTLS between services in production.

### 11. Observability

- **Prometheus** scrapes backend and DB metrics.
- **Grafana** visualizes both SOC and infrastructure KPIs.
- **Loki** aggregates backend logs for correlation with alerts.
- **Alertmanager** triggers Slack or email notifications.

### 12. Scalability and Future Enhancements

- Add **API Gateway (Kong or Traefik Enterprise)** for routing multiple submodules.
- Introduce **message bus (NATS / Kafka)** for cross-system event streams.
- Add **OpenTelemetry** for distributed tracing.
- Integrate **RBAC dashboards** in Keycloak Admin Console.
- Support **plugin modules** for third-party SOC tools.

### 13. Summary

| Goal                      | Technology                  |
|---------------------------|-----------------------------|
| Unified SOC dashboard     | React + NestJS              |
| Secure authentication     | Keycloak (OIDC)             |
| Time-series & config data | PostgreSQL + InfluxDB       |
| Live updates              | WebSockets (Socket.IO)      |
| Monitoring & Logs         | Prometheus + Grafana + Loki |
| Container orchestration   | Docker Compose / Kubernetes |

### 14. Example URL Map

| Endpoint              | Purpose                | Auth Role   |
|-----------------------|------------------------|-------------|
| /api/metrics/overview | System metrics summary | soc_viewer  |
| /api/health           | Service status         | soc_viewer  |
| /api/alerts           | Active alerts feed     | soc_viewer  |
| /api/admin/users      | Manage users           | soc_admin   |
| /api/audit/logs       | View access logs       | soc_manager |

**Version:** 1.0  
**Author:** Systems Architecture Team  
**Maintainer:** alex.campkin@pentanet.io  
**Date:** 2025-11-10

yaml

Copy code

---

Would you like me to include a **draw.io diagram (XML import)** for this architecture (fr